

CREATE – Multi-User SaaS

ExpansionTechnical Blueprint

Document Version: Blueprint v1.0

Target Runtime: Node.js 20 LTS + PostgreSQL 16 on Replit Deployments

Audience: Senior Full-Stack Engineers (Replit)

Date: August 29, 2026

Classification: Internal Engineering Handoff

Table of Contents

- 1. 0. Document Purpose & Scope
- 2. 1. Current Architecture Snapshot 1.1 Single-Operator Baseline 1.2 Key Limitations 1.3 Current Architecture Diagram
- 3. 2. Target Architecture Overview 2.1 Design Pillars 2.2 Target Architecture Diagram
- 4. 3. Workspace & User Data Model 3.1 Conceptual Model 3.2 Roles & Permissions Matrix 3.3 Invitation Flow
- 5. 4. Authentication & Authorization 4.1 Auth Stack Recommendation 4.2 Middleware Chain (Express) 4.3 Token Storage Schema 4.4 Social OAuth Token Vault
- 6. 5. Database Schema Extensions 5.1 Migration Strategy 5.2 Core Schema (PostgreSQL DDL) 5.3 Row-Level Security (RLS) 5.4 Indexes
- 7. 6. API Design & Boundaries 6.1 API Versioning 6.2 Resource Endpoints 6.3 Standard Response Envelope 6.4 Rate Limiting 6.5 WebSocket / Real-Time

8.	7. Scheduling & Publishing Engine	7.1 Architecture	7.2 Job Lifecycle State Machine	7.3 Publisher Interface	7.4 Platform-Specific Implementation Notes	7.5 Retry & Dead Letter Strategy	7.6 Time Zone Handling
9.	8. Media & Export Pipeline	8.1 Upload Flow	8.2 Storage Configuration	8.3 Content Export Pipeline	8.4 Image Optimization		
10.	9. Notifications System	9.1 Notification Types	9.2 Delivery Channels	9.3 Notifications Table Schema			
11.	10. Replit Deployment Guide	10.1 Project Structure	10.2 .replit Configuration	10.3 Environment Variables Reference	10.4 Required Replit Add-ons	10.5 Health Checks & Readiness	10.6 Database Migration on Deploy
		10.7 Worker Process Management	10.8 Secrets Management				
12.	11. Security Hardening Checklist						
13.	12. Migration Path: v1 → v2	12.1 Phase 1 – Foundation (Week 1–2)	12.2 Phase 2 – Isolation (Week 3–4)	12.3 Phase 3 – Publishing Engine (Week 5–6)	12.4 Phase 4 – Polish & Hardening (Week 7–8)	12.5 Rollback Strategy	
14.	13. Testing Strategy	13.1 Unit Tests	13.2 Integration Tests	13.3 End-to-End Tests	13.4 CI on Replit		
15.	Appendix A: Glossary						
16.	Appendix B: Dependency Manifest						

0. Document Purpose & Scope

This blueprint is the authoritative engineering handoff document for transforming CREATE from a single-operator social media content planning application into a production-grade, multi-tenant SaaS platform. Every section contains implementable specifications — schema DDL, TypeScript signatures,

configuration files, and deployment procedures — written to eliminate ambiguity at the point of implementation.

In-Scope

- **Workspace & User Model:** Tenant isolation via workspaces, role-based membership, invitation flow.
- **Authentication & Authorization:** JWT/refresh token stack, OAuth2 per-platform token vault, middleware chain.
- **Database:** Full PostgreSQL 16 DDL, Row-Level Security (RLS) policies, migration strategy, index plan.
- **API Layer:** REST endpoint map, response envelope, rate limiting, WebSocket real-time events.
- **Scheduling Engine:** BullMQ queue architecture, cron scheduler, job lifecycle state machine.
- **Social Publishing:** Platform-specific publisher interface, per-platform implementation notes (Instagram, Facebook, X, LinkedIn, Pinterest, TikTok), retry and dead-letter handling.
- **Media Pipeline:** Presigned upload flow, image/video processing, export formats (PDF, CSV, JSON).
- **Replit Deployment:** Project structure, `.replit` config, environment variables, health checks, worker management, secrets handling.
- **Security:** Full hardening checklist covering transport, storage, injection, OAuth CSRF, and audit logging.
- **Migration Path:** Four-phase v1 → v2 transition plan with rollback strategy.
- **Testing:** Unit, integration, E2E, and CI pipeline specification.

Out-of-Scope (Future Work)

- **Billing & Payments:** Stripe subscription management, plan enforcement beyond feature flags, metered usage — deferred to Billing v1 milestone.
- **White-Labeling:** Custom domain per workspace, branded email senders, theme override API — deferred to Enterprise v1 milestone.

Versioning

Field	Value
Blueprint Version	v1.0
Target Node.js Runtime	20 LTS (iron)
Target Database	PostgreSQL 16
Target Deploy Platform	Replit Deployments (Cloud Run target)
Redis	Upstash Redis (Valkey 7+ compatible)
ORM	Prisma 5.x with raw SQL escape hatches for RLS

1. Current Architecture Snapshot

1.1 Single-Operator Baseline

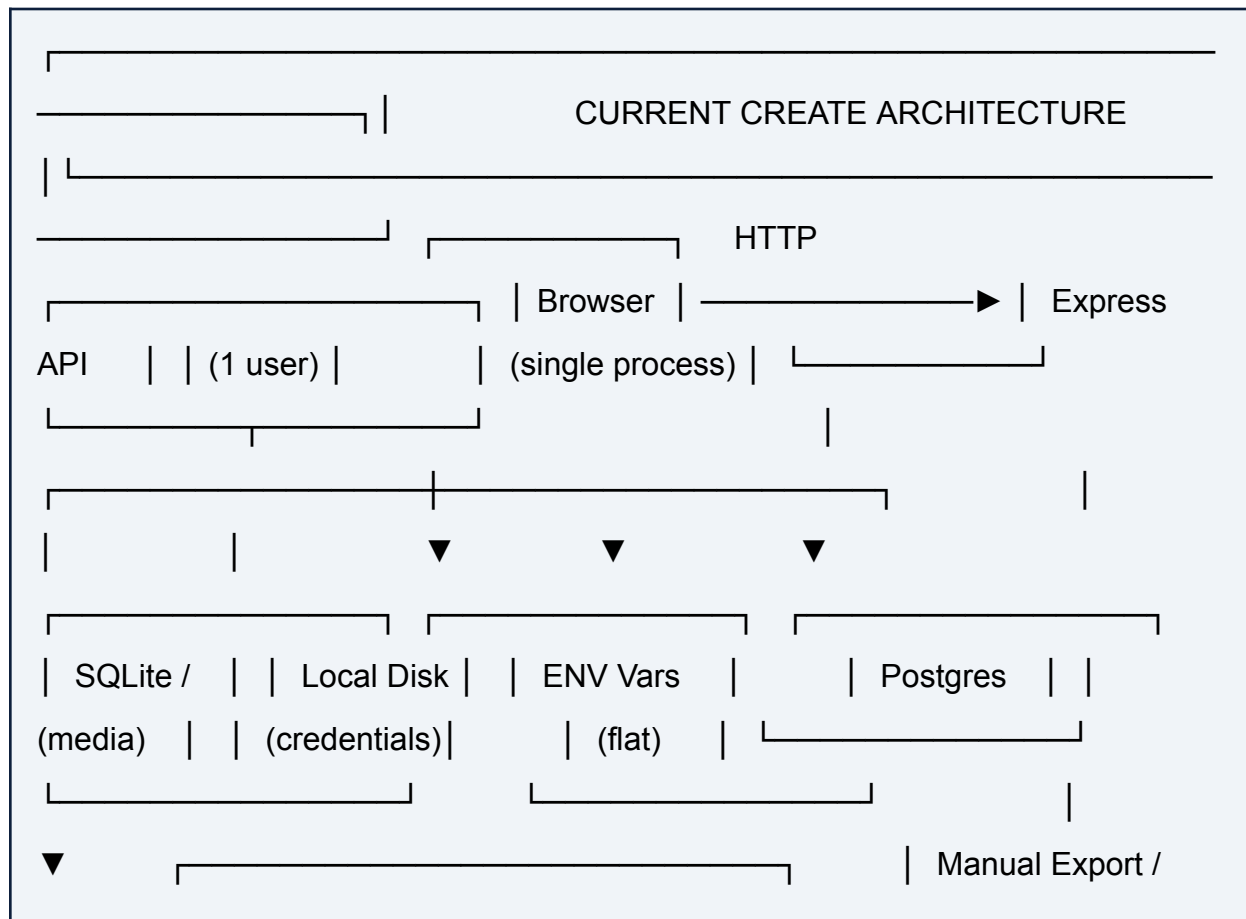
The existing CREATE application is structured as a single-process Node.js/Express server serving one authenticated operator. The data layer uses either SQLite (development) or a single-schema PostgreSQL database with no tenant isolation. Social credentials are stored as plaintext environment variables or a single row in a config table. Content publishing is either manual (copy-paste) or triggered by a single hand-written cron job. Media files are stored on local disk or a single unscoped cloud bucket folder. There is no job queue, no retry logic, and no audit trail.

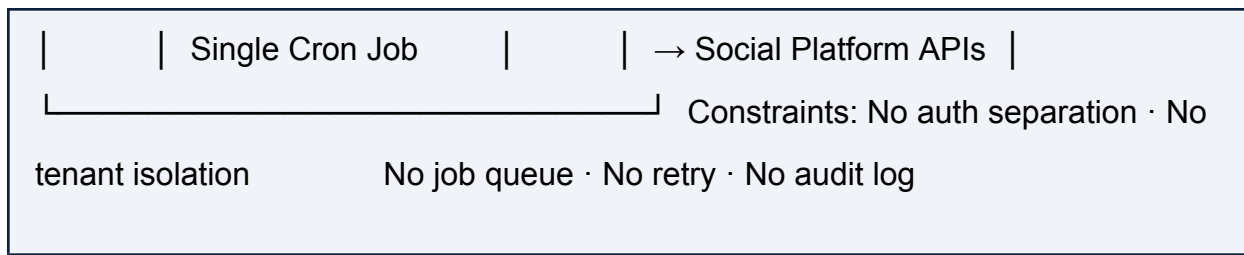
1.2 Key Limitations

- **No row-level isolation:** All data is globally readable by any authenticated session; there is no concept of tenancy.
- **No OAuth token storage per user or team:** Credentials are global, cannot be scoped, rotated per team, or revoked independently.

- **No job queue:** Publishing is synchronous or cron-based with no retry, dead-letter handling, or status tracking.
- **No workspace concept:** The data model has no grouping unit between the platform and individual content items.
- **No role system:** Anyone with the single password can perform any action including deletion and credential changes.
- **No media pipeline:** Files are stored without thumbnail generation, format validation, or platform-specific resizing.
- **No audit trail:** Destructive or privileged actions are unlogged.
- **Local file handling:** Ephemeral Replit filesystem cannot persist large media across deploys.

1.3 Current Architecture Diagram





2. Target Architecture Overview

2.1 Design Pillars

- **Workspace Isolation:** Every data record is scoped to a workspace (one workspace = one team/brand). RLS enforced at the database layer ensures isolation even if the API layer has a bug.
- **Stateless API with Stateful Queue:** The Express API tier is horizontally scalable and stateless. All async work (publishing, media processing, notifications) is delegated to BullMQ workers backed by Redis.
- **Encrypted Credential Store:** Each workspace stores its own OAuth tokens for each connected social platform, encrypted at rest with AES-256-GCM. No credentials are ever stored in environment variables at the workspace level.
- **Durable Object Storage:** Media assets are stored in Cloudflare R2 or Replit Object Storage via the S3-compatible API. No local disk dependency.
- **Real-Time via WebSocket:** Socket.IO rooms scoped to workspace_id push live status events to connected clients without polling.
- **Audit-First:** Every privileged action writes an immutable audit log entry before the action completes.

2.2 Target Architecture Diagram

TARGET CREATE SAAS

ARCHITECTURE

CLIENTS Browser (Next.js/React)

WebSocket (Socket.IO)

HTTPS / WSS

EXPRESS REST API (Node.js 20, /api/v1/)

Helmet / verifyAccess resolveWork CORS Token (JWT) space + RLS

enforceRole Rate Limiter Request (minRole) (Redis store) Validator(Zod)

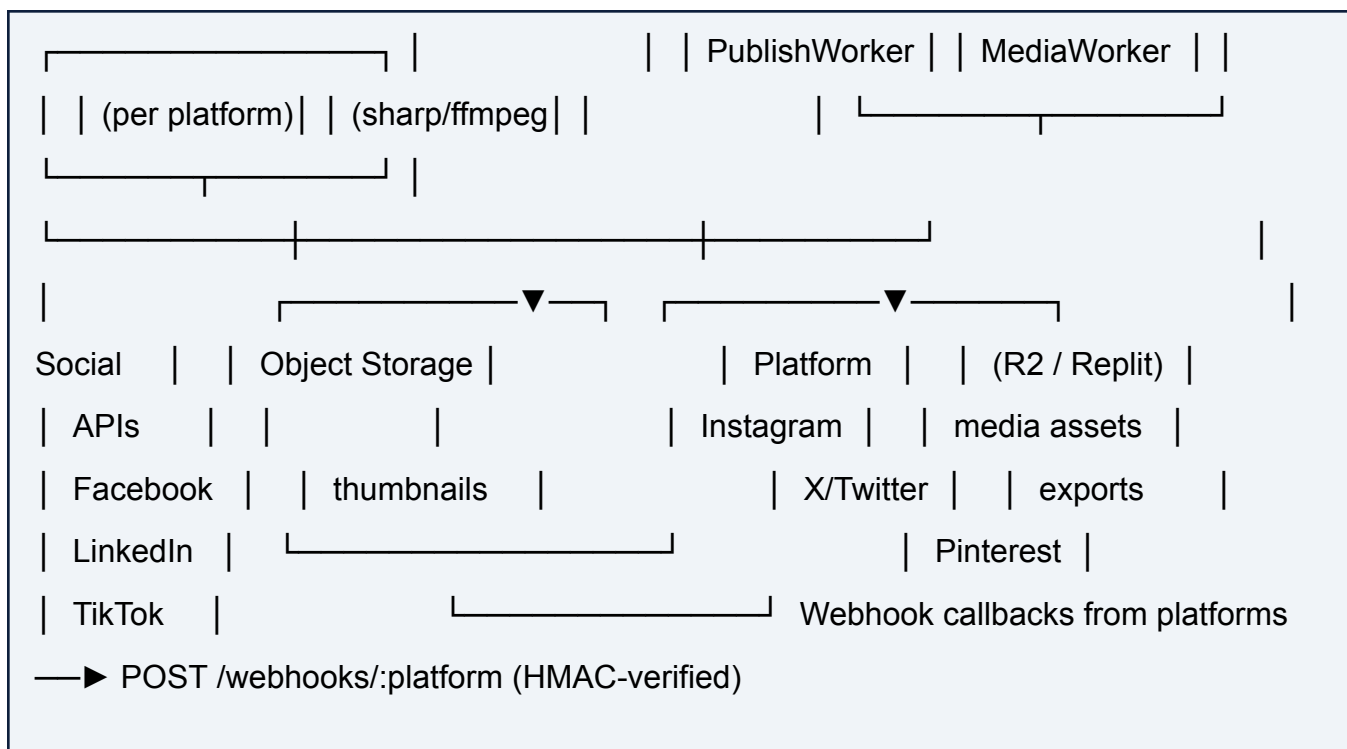
PostgreSQL Redis (Bull) 16 + RLS

Queues: Workspaces content-publish Users media-process Posts notification Jobs

Audit Logs

BULLMQ

WORKERS



3. Workspace & User Data Model

3.1 Conceptual Model

The data model uses a strict three-tier hierarchy. Understanding this hierarchy is required before touching any schema or API code.

Tier	Entity	Visibility	Description
1 – Platform	SaaS System	Internal only	The CREATE platform itself. Exists as the implicit root of all data. Never exposed as an API resource to end users. Relevant only for super-admin tooling and platform-level analytics.

Tier	Entity	Visibility	Description
2 – Workspace	<code>workspaces</code>	Members only	Represents a team or brand account. This is the billing unit. Every piece of content, every social connection, and every media asset belongs to exactly one workspace. A user may belong to many workspaces simultaneously.
3 – User	<code>users</code>	Self + workspace members	An individual human with credentials. Linked to workspaces via <code>workspace_members</code> with an explicit role. A user's role can differ across workspaces.

CRITICAL RULE

Every database table that stores content (posts, media, jobs, campaigns, audit logs, notifications) MUST have a `workspace_id` column with a foreign key to `workspaces.id` and a corresponding RLS policy. No exceptions.

3.2 Roles & Permissions Matrix

Roles are ordered from least to most privileged. Each role inherits all permissions of roles below it, except where noted.

Role	Create Content	Approve / Schedule	Publish Now	Manage Members	Manage Integrations	Admin Settings
Viewer	No	No	No	No	No	No
Creator	Yes (own only)	No	No	No	No	No
Editor	Yes (all)	Limited (schedule only, not approve)	No	No	No	No
Manager	Yes (all)	Yes	Yes	Limited (invite/remove up to Editor)	Yes	No
Owner	Yes (all)	Yes	Yes	Yes (all roles)	Yes	Yes

Role enum order (ascending privilege): `viewer < creator < editor < manager < owner`. Store as a PostgreSQL enum or integer ordinal. The `enforceRole(minRole)` middleware compares the requesting user's role ordinal against the endpoint's required minimum.

3.3 Invitation Flow

The invitation flow is token-based to avoid requiring an existing account before receiving an invite.

1. **Owner or Manager issues invite:** `POST /api/v1/workspaces/:wid/invites` with `{ email, role }`. API validates the inviter has permission to grant the target role.
2. **API generates signed JWT invite token:** Payload: `{ workspaceId, inviteeEmail, role, invitedByUserId, exp: now + 86400 }`. Signed with `JWT_SECRET`. Token is stored (hashed) in an `invites` table with a `used_at` column to prevent replay.
3. **Email delivery:** API enqueues a `notification` job that sends an email via Resend containing: `https://app.create.io/accept-invite?token=<JWT>`.
4. **Recipient clicks link:** Frontend sends `POST /api/v1/auth/accept-invite` with the token. API verifies JWT signature, checks expiry, checks `invites.used_at IS NULL`.
5. **Account creation or linking:** If a user with `inviteeEmail` exists, the flow links the existing account. If not, the recipient is prompted to set a password; a new `users` record is created.

6. **WorkspaceMember record created:** A `workspace_members` row is inserted with the specified role and `joined_at = NOW()`. The `invites.used_at` is stamped. A `member:joined` WebSocket event is emitted to the workspace room.
-

4. Authentication & Authorization

4.1 Auth Stack Recommendation

- **Passport.js** with `passport-local` strategy for email/password, and `passport-google-oauth20` for Google SSO.
- **Access tokens:** JWT, 15-minute expiry, signed with `JWT_SECRET` (HS256). Sent in `Authorization: Bearer <token>` header.
- **Refresh tokens:** Opaque random string (32 bytes, hex), 7-day expiry. Stored as bcrypt hash in `refresh_tokens` table. Delivered as `HttpOnly; Secure; SameSite=Strict` cookie named `__rt`.
- **Refresh token rotation:** Every `POST /auth/refresh` call invalidates the used token and issues a new one. If a revoked token is replayed, all tokens for that user are invalidated (token reuse detection).
- **PKCE:** Required for any public OAuth client (mobile, SPA without backend-for-frontend). Use `code_challenge_method=S256`.

4.2 Middleware Chain (Express)

Every protected route passes through this ordered middleware stack. Middleware is composed via a helper; never apply ad-hoc per route.

```
verifyAccessToken → resolveWorkspace → enforceRole(minRole) → handler
// typescriptimport { Request, Response, NextFunction } from 'express';import
jwt from 'jsonwebtoken';import { prisma } from '../db/client';export interface
AuthenticatedRequest extends Request { user: { id: string; email:
string; }; workspace: { id: string; slug: string; plan: 'free' |
'pro' | 'team'; }; memberRole: 'viewer' | 'creator' | 'editor' | 'manager' |
'owner';}// --- Middleware 1: Verify JWT access token ---export function
verifyAccessToken( req: Request, res: Response, next: NextFunction): void {
```

```

const authHeader = req.headers['authorization']; if
(!authHeader?.startsWith('Bearer ')) { res.status(401).json({ errors: [{
code: 'UNAUTHORIZED', message: 'Missing access token' }] }); return; }
const token = authHeader.slice(7); try { const payload = jwt.verify(token,
process.env.JWT_SECRET!) as { sub: string; email: string; };
(req as AuthenticatedRequest).user = { id: payload.sub, email: payload.email };
next(); } catch (err) { res.status(401).json({ errors: [{ code:
'TOKEN_INVALID', message: 'Access token invalid or expired' }] }); } // ---
Middleware 2: Resolve workspace + member role ---export async function
resolveWorkspace( req: Request, res: Response, next: NextFunction): Promise
{ const authReq = req as AuthenticatedRequest; const workspaceId =
req.params.wid ?? req.body?.workspaceId; if (!workspaceId) {
res.status(400).json({ errors: [{ code: 'BAD_REQUEST', message: 'Workspace ID
required' }] }); return; } const membership = await
prisma.workspaceMember.findUnique({ where: { workspace_id_user_id: {
workspace_id: workspaceId, user_id: authReq.user.id, }, },
include: { workspace: true }, }); if (!membership || membership.left_at !==
null) { res.status(403).json({ errors: [{ code: 'FORBIDDEN', message: 'Not a
member of this workspace' }] }); return; } authReq.workspace = { id:
membership.workspace.id, slug: membership.workspace.slug, plan:
membership.workspace.plan, }; authReq.memberRole = membership.role as
AuthenticatedRequest['memberRole']; // Set RLS context variable for this
connection await prisma.$executeRawUnsafe( `SET LOCAL
app.current_workspace_id = '${workspaceId}'` ); next(); } // --- Middleware 3:
Enforce minimum role ---const ROLE_ORDER = ['viewer', 'creator', 'editor',
'manager', 'owner'] as const; type Role = typeof ROLE_ORDER[number]; export
function enforceRole(minRole: Role) { return (req: Request, res: Response,
next: NextFunction): void => { const authReq = req as AuthenticatedRequest;
const userRoleIndex = ROLE_ORDER.indexOf(authReq.memberRole); const
minRoleIndex = ROLE_ORDER.indexOf(minRole); if (userRoleIndex <
minRoleIndex) { res.status(403).json({ errors: [{ code: 'FORBIDDEN',
message: `Requires role: ${minRole}` }] }); return; } next(); }; }

```

4.3 Token Storage Schema

```

-- sqlCREATE TABLE refresh_tokens ( id UUID PRIMARY KEY DEFAULT
gen_random_uuid(), user_id UUID NOT NULL REFERENCES users(id) ON DELETE
CASCADE, token_hash TEXT NOT NULL, -- bcrypt hash of the
raw token workspace_id UUID REFERENCES workspaces(id) ON DELETE CASCADE,
expires_at TIMESTAMPTZ NOT NULL, revoked_at TIMESTAMPTZ,
-- NULL = active created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()); CREATE
INDEX idx_refresh_tokens_user_id ON refresh_tokens(user_id); CREATE INDEX
idx_refresh_tokens_expires_at ON refresh_tokens(expires_at) WHERE revoked_at
IS NULL;

```

4.4 Social OAuth Token Vault

Each workspace stores one `social_connections` row per connected social platform. Access and refresh tokens are encrypted at rest using AES-256-GCM before writing to the database. The raw token never touches the DB layer.

```

-- sqlCREATE TYPE social_platform AS ENUM ( 'instagram', 'facebook',
'x_twitter', 'linkedin', 'pinterest', 'tiktok'); CREATE TABLE
social_connections ( id UUID PRIMARY KEY DEFAULT
gen_random_uuid(), workspace_id UUID NOT NULL REFERENCES

```

```
workspaces(id) ON DELETE CASCADE, platform social_platform NOT
NULL, access_token TEXT NOT NULL, -- AES-256-GCM
encrypted, base64 refresh_token TEXT, --
AES-256-GCM encrypted, base64; NULL if not issued token_expires_at
TIMESTAMPTZ, scopes TEXT[] NOT NULL DEFAULT '{}', key_version
SMALLINT NOT NULL DEFAULT 1, -- for key rotation connected_by_user_id UUID NOT
NULL REFERENCES users(id), created_at TIMESTAMPTZ NOT NULL DEFAULT
NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE(workspace_id, platform));CREATE INDEX idx_social_connections_workspace
ON social_connections(workspace_id);CREATE INDEX idx_social_connections_expiry
ON social_connections(token_expires_at) WHERE token_expires_at IS NOT NULL;
```

Encryption Key Management: Set `SOCIAL_TOKEN_ENCRYPTION_KEY` as a 32-byte base64-encoded

secret in Replit Secrets. The application loads this at startup, validates its length, and derives an

AES-256-GCM cipher. To rotate keys: increment `key_version`, add the new key as

`SOCIAL_TOKEN_ENCRYPTION_KEY_V2`, run a background job that re-encrypts all rows from version 1 to

version 2, then remove the old key. The `key_version` column tells the decryption routine which key version to use.

```
// typescriptimport { createCipheriv, createDecipheriv, randomBytes } from
'crypto';const ALGORITHM = 'aes-256-gcm';const KEY =
Buffer.from(process.env.SOCIAL_TOKEN_ENCRYPTION_KEY!, 'base64');export function
encryptToken(plaintext: string): string { const iv = randomBytes(12); const
cipher = createCipheriv(ALGORITHM, KEY, iv); const encrypted =
Buffer.concat([cipher.update(plaintext, 'utf8'), cipher.final()]); const tag =
cipher.getAuthTag(); // Format: iv(12b):tag(16b):ciphertext - all base64
return Buffer.concat([iv, tag, encrypted]).toString('base64');}export function
decryptToken(encoded: string): string { const buf = Buffer.from(encoded,
'base64'); const iv = buf.slice(0, 12); const tag = buf.slice(12, 28); const
ciphertext = buf.slice(28); const decipher = createDecipheriv(ALGORITHM, KEY,
iv); decipher.setAuthTag(tag); return
Buffer.concat([decipher.update(ciphertext),
decipher.final()]).toString('utf8');}
```

5. Database Schema Extensions

5.1 Migration Strategy

- **Tool:** Prisma Migrate for schema-driven migrations; raw SQL escape hatches via `prisma.$executeRaw` for RLS policy DDL (Prisma does not manage RLS natively).
- **Additive-only rule:** No migration may DROP a column or TABLE in a single step that hasn't been deprecated in the previous release. Breaking changes follow: dual-write (new + old column both written) → backfill job → read from new column → drop old in subsequent release.

- **File naming:** `YYYYMMDDHHMMSS_description.sql` (e.g., `20260829120000_add_workspace_members.sql`).
- **Transaction guarantee:** Each migration file is wrapped in a transaction. Failure rolls back the entire migration, never leaving schema in a partial state.
- **Down migrations:** Every migration has a corresponding rollback script in `packages/db/migrations/down/`.

5.2 Core Schema (PostgreSQL DDL)

```
-- sql-- Enable required extensionsCREATE EXTENSION IF NOT EXISTS
"pgcrypto";CREATE EXTENSION IF NOT EXISTS "pg_trgm"; -- for text search--
-- ENUM TYPES--
CREATE TYPE workspace_plan AS ENUM
('free', 'pro', 'team');CREATE TYPE member_role AS ENUM ('viewer', 'creator',
'editor', 'manager', 'owner');CREATE TYPE post_status AS ENUM ('draft',
'review', 'approved', 'scheduled', 'published', 'failed');CREATE TYPE
job_status AS ENUM ('pending', 'processing', 'success', 'failed',
'cancelled');-- WORKSPACES--
CREATE TABLE workspaces ( id
UUID PRIMARY KEY DEFAULT gen_random_uuid(), name TEXT NOT NULL,
slug TEXT NOT NULL, -- URL-safe unique identifier plan
workspace_plan NOT NULL DEFAULT 'free', owner_user_id UUID NOT NULL,
-- FK added after users table default_timezone TEXT NOT NULL DEFAULT 'UTC',
-- IANA tz string webhook_secret TEXT, -- HMAC secret
for outbound webhooks created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), deleted_at TIMESTAMPTZ,
-- soft delete CONSTRAINT workspaces_slug_unique UNIQUE (slug), CONSTRAINT
workspaces_slug_format CHECK (slug ~ '^[-a-z0-9-]{3,63}$');CREATE INDEX
idx_workspaces_slug ON workspaces(slug);CREATE INDEX idx_workspaces_owner ON
workspaces(owner_user_id);-- USERS--
CREATE TABLE users ( id
UUID PRIMARY KEY DEFAULT gen_random_uuid(), email TEXT NOT NULL,
password_hash TEXT, -- NULL for OAuth-only accounts
display_name TEXT NOT NULL, avatar_url TEXT,
email_verified_at TIMESTAMPTZ, last_login_at TIMESTAMPTZ, created_at
TIMESTAMPTZ NOT NULL DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL
DEFAULT NOW(), CONSTRAINT users_email_unique UNIQUE (email), CONSTRAINT
users_email_format CHECK (email ~* '^[^@]+@[^@]+\.[^@]+$');CREATE INDEX
idx_users_email ON users(email);-- Add FK now that users existsALTER TABLE
workspaces ADD CONSTRAINT fk_workspaces_owner FOREIGN KEY (owner_user_id)
REFERENCES users(id);-- WORKSPACE MEMBERS--
CREATE TABLE
workspace_members ( id
UUID PRIMARY KEY DEFAULT gen_random_uuid(),
workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, role
member_role NOT NULL DEFAULT 'viewer', invited_by UUID REFERENCES
users(id), joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), left_at
TIMESTAMPTZ, CONSTRAINT workspace_members_unique UNIQUE (workspace_id,
user_id);CREATE INDEX idx_wm_workspace ON
workspace_members(workspace_id);CREATE INDEX idx_wm_user ON
workspace_members(user_id);-- CAMPAIGNS--
CREATE TABLE campaigns (
```

```

id          UUID PRIMARY KEY DEFAULT gen_random_uuid(), workspace_id  UUID
NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE, name          TEXT NOT
NULL, description    TEXT, start_date    DATE, end_date            DATE,
created_by          UUID NOT NULL REFERENCES users(id), created_at    TIMESTAMPTZ
NOT NULL DEFAULT NOW(), updated_at      TIMESTAMPTZ NOT NULL DEFAULT
NOW());CREATE INDEX idx_campaigns_workspace ON campaigns(workspace_id);--
-- CONTENT POSTS--
CREATE TABLE content_posts ( id
UUID PRIMARY KEY DEFAULT gen_random_uuid(), workspace_id  UUID NOT NULL
REFERENCES workspaces(id) ON DELETE CASCADE, created_by      UUID NOT NULL
REFERENCES users(id), title          TEXT NOT NULL, body_json
JSONB NOT NULL DEFAULT '{}', -- rich text / editor.js format status
post_status NOT NULL DEFAULT 'draft', scheduled_at          TIMESTAMPTZ,
published_at      TIMESTAMPTZ, target_platforms TEXT[] NOT NULL DEFAULT '{}',
-- ['instagram','linkedin',...] media_asset_ids  UUID[] NOT NULL DEFAULT
 '{}', tags        TEXT[] NOT NULL DEFAULT '{}', campaign_id  UUID
REFERENCES campaigns(id) ON DELETE SET NULL, created_at      TIMESTAMPTZ NOT
NULL DEFAULT NOW(), updated_at      TIMESTAMPTZ NOT NULL DEFAULT
NOW());CREATE INDEX idx_posts_workspace_status_scheduled ON
content_posts(workspace_id, status, scheduled_at);CREATE INDEX
idx_posts_campaign ON content_posts(campaign_id) WHERE campaign_id IS NOT
NULL;CREATE INDEX idx_posts_tags ON content_posts USING GIN (tags);CREATE INDEX
idx_posts_body ON content_posts USING GIN (body_json);--
-- MEDIA ASSETS--
CREATE TABLE media_assets ( id
UUID PRIMARY KEY DEFAULT gen_random_uuid(), workspace_id  UUID NOT NULL
REFERENCES workspaces(id) ON DELETE CASCADE, uploaded_by      UUID NOT NULL
REFERENCES users(id), filename          TEXT NOT NULL, mime_type    TEXT
NOT NULL, size_bytes          BIGINT NOT NULL, storage_key      TEXT NOT NULL,
-- R2/object-storage path thumbnail_key    TEXT, width          INT,
height          INT, duration_seconds NUMERIC(10,3),          -- for video
assets created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(), CONSTRAINT
media_assets_positive_size CHECK (size_bytes > 0));CREATE INDEX
idx_media_workspace ON media_assets(workspace_id);--
-- PUBLISH JOBS--
CREATE TABLE publish_jobs ( id
UUID PRIMARY KEY DEFAULT gen_random_uuid(), content_post_id  UUID NOT NULL
REFERENCES content_posts(id) ON DELETE CASCADE, workspace_id  UUID NOT
NULL REFERENCES workspaces(id) ON DELETE CASCADE, platform      TEXT NOT
NULL, status            job_status NOT NULL DEFAULT 'pending', scheduled_at
TIMESTAMPTZ NOT NULL, started_at      TIMESTAMPTZ, completed_at
TIMESTAMPTZ, platform_post_id TEXT,          -- ID returned by
social platform API error_message    TEXT, retry_count        SMALLINT NOT
NULL DEFAULT 0, created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW());CREATE
INDEX idx_jobs_workspace_platform_status ON publish_jobs(workspace_id,
platform, status);CREATE INDEX idx_jobs_post ON
publish_jobs(content_post_id);CREATE INDEX idx_jobs_scheduled ON
publish_jobs(scheduled_at) WHERE status = 'pending';--
-- AUDIT LOGS--
CREATE TABLE audit_logs ( id
UUID PRIMARY KEY DEFAULT gen_random_uuid(), workspace_id  UUID NOT NULL
REFERENCES workspaces(id) ON DELETE CASCADE, actor_user_id  UUID REFERENCES
users(id) ON DELETE SET NULL, action          TEXT NOT NULL,          --
e.g., 'post.approved', 'member.removed' resource_type TEXT NOT NULL,
-- e.g., 'content_post', 'workspace_member' resource_id  UUID, diff
JSONB,          -- JSON patch of before/after state ip_address
INET, created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW());-- Audit logs are
append-only; no UPDATE/DELETE on this table ever.CREATE INDEX
idx_audit_workspace_created ON audit_logs(workspace_id, created_at DESC);CREATE
INDEX idx_audit_actor ON audit_logs(actor_user_id);

```

5.3 Row-Level Security (RLS)

RLS is the last line of defence ensuring no API bug can leak cross-workspace data. All application queries run as a low-privilege DB role (`app_user`) which has no bypass privilege. The API sets a session-local variable before every query batch.

```
-- sql-- Create low-privilege application roleCREATE ROLE app_user LOGIN
PASSWORD '<generated-strong-password>';GRANT CONNECT ON DATABASE create_db TO
app_user;GRANT USAGE ON SCHEMA public TO app_user;GRANT SELECT, INSERT, UPDATE,
DELETE ON ALL TABLES IN SCHEMA public TO app_user;GRANT USAGE, SELECT ON ALL
SEQUENCES IN SCHEMA public TO app_user;-- Enable RLS on workspace-scoped
tablesALTER TABLE content_posts ENABLE ROW LEVEL SECURITY;ALTER TABLE
media_assets ENABLE ROW LEVEL SECURITY;ALTER TABLE publish_jobs ENABLE ROW
LEVEL SECURITY;ALTER TABLE campaigns ENABLE ROW LEVEL SECURITY;ALTER TABLE
audit_logs ENABLE ROW LEVEL SECURITY;ALTER TABLE social_connections ENABLE ROW
LEVEL SECURITY;ALTER TABLE workspace_members ENABLE ROW LEVEL SECURITY;ALTER
TABLE notifications ENABLE ROW LEVEL SECURITY;-- — RLS Policies: content_posts
—CREATE POLICY content_posts_workspace_isolation ON content_posts AS
PERMISSIVE FOR ALL TO app_user USING (workspace_id =
current_setting('app.current_workspace_id', true)::UUID) WITH CHECK
(workspace_id = current_setting('app.current_workspace_id', true)::UUID);-- —
RLS Policies: media_assets —CREATE POLICY media_assets_workspace_isolation ON
media_assets AS PERMISSIVE FOR ALL TO app_user USING (workspace_id =
current_setting('app.current_workspace_id', true)::UUID) WITH CHECK
(workspace_id = current_setting('app.current_workspace_id', true)::UUID);--
Apply equivalent policies to all other workspace-scoped tables-- (publish_jobs,
campaigns, audit_logs, social_connections,-- workspace_members, notifications)
— follow the same pattern above.
```

How the API sets the RLS context: In the `resolveWorkspace` middleware (Section 4.2), after confirming workspace membership, the middleware executes:

```
-- sqlSET LOCAL app.current_workspace_id = '<uuid>';
```

`SET LOCAL` scopes the variable to the current transaction. Because Prisma uses connection pooling, always set this within an explicit transaction block or use `PgBouncer` in transaction mode. Never use `SET` (session-scoped) as it persists across pooled connections.

5.4 Indexes

Table	Index Type	Columns	Condition	Purpose
content_posts	B-tree composite	(workspace_id, status, scheduled_at)	—	Scheduler polling query
content_posts	GIN	tags	—	Tag-based filtering
content_posts	GIN	body_json	—	Full-text JSONB search

Table	Index Type	Columns	Condition	Purpose
publish_jobs	B-tree composite	(workspace_id, platform, status)	—	Job dashboard queries
publish_jobs	B-tree partial	scheduled_at	status = 'pending'	Scheduler pickup
media_assets	B-tree	workspace_id	—	Media library listing
audit_logs	B-tree composite	(workspace_id, created_at DESC)	—	Audit log pagination
refresh_tokens	B-tree partial	expires_at	revoked_at IS NULL	Token cleanup cron
social_connections	B-tree partial	token_expires_at	token_expires_at IS NOT NULL	Token refresh scheduler
users	B-tree	email	—	Login lookup
workspaces	B-tree	slug	—	Slug resolution

6. API Design & Boundaries

6.1 API Versioning

All endpoints are prefixed with `/api/v1/`. Future API versions negotiate via the `Accept` header:
`Accept: application/vnd.create.v2+json`. Version 1 endpoints remain stable for a minimum of 12 months after a v2 is published. Never rename or remove v1 endpoints without a deprecation notice period.

6.2 Resource Endpoints (Complete REST Map)

Method	Path	Auth	Min Role	Description
Authentication				
POST	/api/v1/auth/register	No	—	Create a new user account

Method	Path	Auth	Min Role	Description
POST	/api/v1/auth/login	No	—	Authenticate; returns access token + sets refresh cookie
POST	/api/v1/auth/refresh	Cookie	—	Exchange refresh token for new access + refresh token pair
POST	/api/v1/auth/logout	Bearer	—	Revoke current refresh token
GET	/api/v1/auth/me	Bearer	—	Return authenticated user profile
POST	/api/v1/auth/accept-i nvite	No	—	Accept workspace invite token
GET	/api/v1/auth/google	No	—	Initiate Google OAuth2 PKCE flow
GET	/api/v1/auth/google/c allback	No	—	Google OAuth2 callback handler
Workspaces				
POST	/api/v1/workspaces	Bearer	—	Create workspace; creator becomes Owner
GET	/api/v1/workspaces/: wid	Bearer	Viewer	Get workspace details
PATCH	/api/v1/workspaces/: wid	Bearer	Owner	Update workspace name, slug, timezone
DELETE	/api/v1/workspaces/: wid	Bearer	Owner	Soft-delete workspace (sets deleted_at)
Members & Invites				
GET	/api/v1/workspaces/: wid/members	Bearer	Viewer	List all workspace members with roles
POST	/api/v1/workspaces/: wid/invites	Bearer	Manager	Send invite email to a new member
GET	/api/v1/workspaces/: wid/invites	Bearer	Manager	List pending invites

Method	Path	Auth	Min Role	Description
DELETE	/api/v1/workspaces/:wid/invites/:iid	Bearer	Manager	Revoke pending invite
PATCH	/api/v1/workspaces/:wid/members/:uid	Bearer	Owner	Change member role
DELETE	/api/v1/workspaces/:wid/members/:uid	Bearer	Manager	Remove member from workspace
Content Posts				
GET	/api/v1/workspaces/:wid/posts	Bearer	Viewer	List posts with filters (status, platform, tags, date range, campaign)
POST	/api/v1/workspaces/:wid/posts	Bearer	Creator	Create a new content post (draft)
GET	/api/v1/workspaces/:wid/posts/:pid	Bearer	Viewer	Get full post including body_json and media
PATCH	/api/v1/workspaces/:wid/posts/:pid	Bearer	Creator	Update post body, title, tags, campaign
DELETE	/api/v1/workspaces/:wid/posts/:pid	Bearer	Editor	Delete post (only if draft/review)
POST	/api/v1/workspaces/:wid/posts/:pid/approve	Bearer	Manager	Set status to approved
POST	/api/v1/workspaces/:wid/posts/:pid/schedule	Bearer	Editor	Set scheduled_at; status → scheduled
POST	/api/v1/workspaces/:wid/posts/:pid/publish-now	Bearer	Manager	Immediately enqueue publish job for all target_platforms
POST	/api/v1/workspaces/:wid/posts/:pid/duplicate	Bearer	Creator	Clone post as new draft
POST	/api/v1/workspaces/:wid/posts/:pid/unschedule	Bearer	Editor	Cancel scheduled post; status → approved
Campaigns				
GET	/api/v1/workspaces/:wid/campaigns	Bearer	Viewer	List campaigns

Method	Path	Auth	Min Role	Description
POST	/api/v1/workspaces/:wid/campaigns	Bearer	Editor	Create campaign
GET	/api/v1/workspaces/:wid/campaigns/:cid	Bearer	Viewer	Get campaign + associated posts
PATCH	/api/v1/workspaces/:wid/campaigns/:cid	Bearer	Editor	Update campaign metadata
DELETE	/api/v1/workspaces/:wid/campaigns/:cid	Bearer	Manager	Delete campaign (posts set campaign_id = NULL)
Media Assets				
GET	/api/v1/workspaces/:wid/media	Bearer	Viewer	List media assets with pagination
POST	/api/v1/workspaces/:wid/media/presign	Bearer	Creator	Request presigned upload URL
POST	/api/v1/workspaces/:wid/media/:id/confirm	Bearer	Creator	Confirm upload complete; enqueue media-process job
GET	/api/v1/workspaces/:wid/media/:id	Bearer	Viewer	Get asset metadata
DELETE	/api/v1/workspaces/:wid/media/:id	Bearer	Editor	Delete asset from storage + DB
Social Integrations				
GET	/api/v1/workspaces/:wid/integrations	Bearer	Viewer	List connected platforms (tokens redacted)
GET	/api/v1/workspaces/:wid/integrations/:platform/connect	Bearer	Manager	Initiate OAuth2 authorization code flow
GET	/api/v1/workspaces/:wid/integrations/:platform/callback	No	—	OAuth2 callback; stores encrypted tokens
DELETE	/api/v1/workspaces/:wid/integrations/:platform	Bearer	Manager	Disconnect platform; delete tokens
Publish Jobs				

Method	Path	Auth	Min Role	Description
GET	/api/v1/workspaces/:wid/jobs	Bearer	Viewer	List jobs with filters (status, platform, date)
GET	/api/v1/workspaces/:wid/jobs/:jid	Bearer	Viewer	Get job detail including error_message
POST	/api/v1/workspaces/:wid/jobs/:jid/retry	Bearer	Manager	Reset retry_count=0, status=pending; re-enqueue
POST	/api/v1/workspaces/:wid/jobs/:jid/cancel	Bearer	Manager	Cancel pending job; status → cancelled
Audit Logs				
GET	/api/v1/workspaces/:wid/audit-logs	Bearer	Manager	List audit entries with filters (actor, action, resource, date range)
Webhooks (Inbound from Platforms)				
POST	/api/v1/webhooks/:platform	None (HMAC)	—	Receive platform callbacks; verified by HMAC-SHA256
Notifications				
GET	/api/v1/workspaces/:wid/notifications	Bearer	Viewer	List notifications for current user (paginated)
POST	/api/v1/workspaces/:wid/notifications/read-all	Bearer	Viewer	Mark all notifications as read
PATCH	/api/v1/workspaces/:wid/notifications/:nid	Bearer	Viewer	Mark single notification read
System				
GET	/health	No	—	Liveness check (always 200 if process alive)
GET	/ready	No	—	Readiness check (DB + Redis ping)

6.3 Standard Response Envelope

```
// json{  "data": {      // Single object or array of objects  },  "meta": {
"page": 1,      "per_page": 25,      "total": 143,      "total_pages": 6  },
"errors": [      {      "code": "VALIDATION_ERROR",      "message": "scheduled_at
must be a future timestamp",      "field": "scheduled_at"      }  ]}
```

Rules: `data` is present on success (may be `null` for 204). `errors` is present on 4xx/5xx (may be an empty array on 200). `meta` is present only on list endpoints. HTTP status codes are the source of truth for success/failure — never bury errors in a 200 response.

6.4 Rate Limiting

```
// typescriptimport rateLimit from 'express-rate-limit';import RedisStore from
'rate-limit-redis';import { redis } from '../lib/redis';const baseConfig = {
windowMs: 60 * 1000, // 1 minute  standardHeaders: true,  legacyHeaders: false,
handler: (req, res) => {      res.status(429).json({      errors: [{ code:
'RATE_LIMITED', message: 'Too many requests' }]}      });  },  store: new
RedisStore({ sendCommand: (...args) => redis.call(...args) }));export const
anonymousLimiter = rateLimit({ ...baseConfig, max: 20 });export const
authenticatedLimiter = rateLimit({ ...baseConfig, max: 200 });export const
mediaUploadLimiter = rateLimit({ ...baseConfig, max: 10 });export const
authLimiter = rateLimit({ ...baseConfig, max: 5 }); // /auth/login only
```

The `Retry-After` header (seconds until next allowed request) is included in all 429 responses via `standardHeaders: true`. Rate limit keys are scoped by IP for anonymous requests and by `user_id + IP` for authenticated requests.

6.5 WebSocket / Real-Time

```
// typescriptimport { Server } from 'socket.io';import jwt from
'jsonwebtoken';const io = new Server(httpServer, { cors: { origin:
process.env.FRONTEND_URL, credentials: true },});// Auth middleware: validate
JWT on handshakeio.use((socket, next) => {  const token =
socket.handshake.auth?.token;  if (!token) return next(new
Error('Authentication required'));  try {    const payload = jwt.verify(token,
process.env.JWT_SECRET!) as { sub: string };    socket.data.userId =
payload.sub;    next();  } catch {    next(new Error('Invalid token'));  }
});io.on('connection', (socket) => {  // Client must join workspace room after
connecting  socket.on('join:workspace', (workspaceId: string) => {    // Verify
membership server-side before allowing join    socket.join(`workspace:${workspaceId}`);  });});// Emit helpers used by API
handlers and workers:export const emitToWorkspace = (workspaceId: string,
event: string, payload: unknown) =>
io.to(`workspace:${workspaceId}`).emit(event, payload);
```

Event Name	Emitted When	Payload Keys
post:updated	Any field of a <code>content_post</code> changes	<code>postId</code> , <code>workspaceId</code> , changes

Event Name	Emitted When	Payload Keys
post:status_changed	Status transitions (draft→review, etc.)	postId, oldStatus, newStatus
job:progress	Publish job starts, succeeds, or fails	jobId, postId, platform, status, error?
member:joined	Invite accepted	userId, displayName, role
notification:new	New notification created for user	notification (full object)
integration:connected	Social platform OAuth completed	platform, connectedByUserId

7. Scheduling & Publishing Engine

7.1 Architecture

Three named BullMQ queues handle all async work. Each queue has dedicated worker instances. The cron scheduler is a lightweight `node-cron` process that only enqueues jobs — it does no publishing itself.

Queue Name	Concurrency	Purpose	Worker File
content-publish	5 per worker instance	Call social platform APIs to publish content	workers/publishWorker.ts
media-process	3 per worker instance	Generate thumbnails, extract metadata, resize	workers/mediaWorker.ts

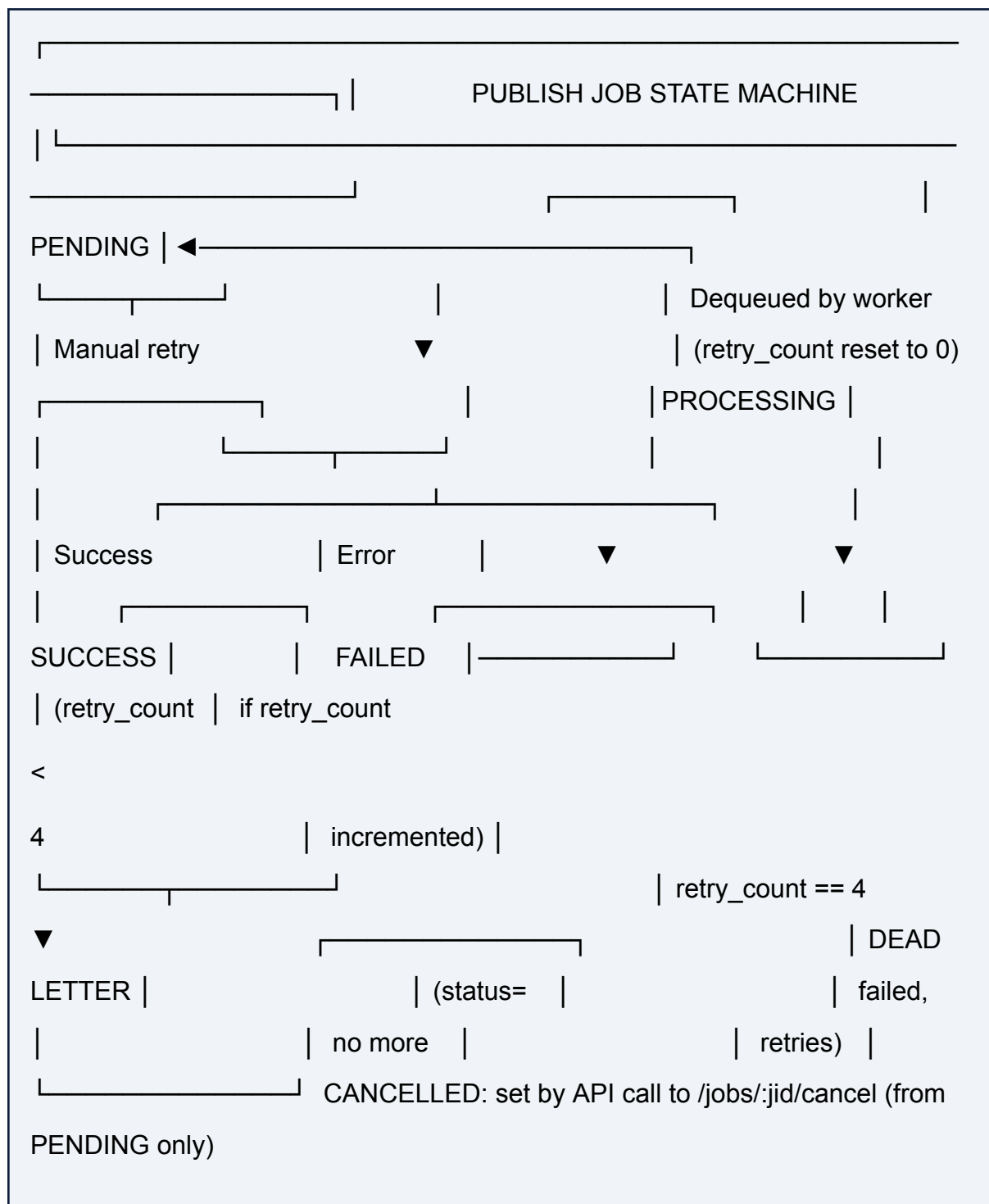
Queue Name	Concurrency	Purpose	Worker File
notification	10 per worker instance	Send emails, store in-app notifications	workers/notification Worker.ts

```
// typescript// packages/queue/src/publishWorker.tsimport { Worker, Job } from 'bullmq';import { connection } from '../redisConnection';import { getPublisher } from '../publishers';import { prisma } from '@create/db';export const publishWorker = new Worker( 'content-publish', async (job: Job<PublishJobPayload>) => {    const { publishJobId } = job.data;    // Mark as processing    await prisma.publishJob.update({        where: { id: publishJobId },        data: { status: 'processing', started_at: new Date() },    });    const publishJob = await prisma.publishJob.findUniqueOrThrow({        where: { id: publishJobId },        include: { content_post: true },    });    const connection = await prisma.socialConnection.findUniqueOrThrow({        where: {            workspace_id_platform: {                workspace_id: publishJob.workspace_id,                platform: publishJob.platform as any,            },        });    const publisher = getPublisher(publishJob.platform);    const tokens = await publisher.refreshTokenIfNeeded(connection);    const result = await publisher.publish(publishJob, publishJob.content_post, tokens);    await prisma.publishJob.update({        where: { id: publishJobId },        data: {            status: 'success',            completed_at: new Date(),            platform_post_id: result.platformPostId,        },    });    const { connection, concurrency: 5, defaultJobOptions: { attempts: 4, backoff: { type: 'exponential', delay: 30000 }, // 30s base },    });publishWorker.on('failed', async (job, err) => {    if (!job) return;    const isFinalAttempt = job.attemptsMade >= (job.opts.attempts ?? 4);    await prisma.publishJob.update({        where: { id: job.data.publishJobId },        data: {            status: isFinalAttempt ? 'failed' : 'pending',            error_message: err.message,            retry_count: job.attemptsMade,        },    });});
```

Scheduler cron job — runs every minute:

```
// typescriptimport cron from 'node-cron';import { publishQueue } from '../queues';import { prisma } from '@create/db';cron.schedule('* * * * *', async () => {    const duePosts = await prisma.contentPost.findMany({        where: {            status: 'scheduled',            scheduled_at: { lte: new Date(Date.now() + 60_000) },        },    });    for (const post of duePosts) {        for (const platform of post.target_platforms) {            // Create publish_jobs record then enqueue            const job = await prisma.publishJob.create({                data: {                    content_post_id: post.id,                    workspace_id: post.workspace_id,                    platform,                    status: 'pending',                    scheduled_at: post.scheduled_at!,                },            });            await publishQueue.add('publish', {                publishJobId: job.id }, {                jobId: job.id, // idempotency key                delay: Math.max(0, post.scheduled_at!.getTime() - Date.now()),            }, {                // Update post status to prevent re-pickup                await prisma.contentPost.update({                    where: { id: post.id },                    data: { status: 'published' }, // set to in-flight status                });            });        }    });
```

7.2 Job Lifecycle State Machine



7.3 Publisher Interface

Every platform publisher is a class implementing `SocialPublisher`. The worker calls only this interface — platform-specific code is fully encapsulated. Adding a new platform requires only a new class implementing this interface and a registration entry in `getPublisher()`.

```
// typescriptexport type PlatformEnum = | 'instagram' | 'facebook' |
'x_twitter' | 'linkedin' | 'pinterest' | 'tiktok';export interface OAuthTokens
{ accessToken: string; refreshToken?: string; expiresAt?: Date; scopes:
string[];}export interface PlatformResult { platformPostId: string; url?:
string; metadata?: Record<string, unknown>;}export interface ValidationResult
{ valid: boolean; errors: Array<{ field: string; message: string }>;}export
interface SocialPublisher { readonly platform: PlatformEnum; /** * Publish
content to the platform. * Must be idempotent – check platform_post_id before
re-publishing. */ publish( job: PublishJob, post: ContentPost,
tokens: OAuthTokens ): Promise<PlatformResult>; /** * Check token expiry;
refresh if needed. * Must update social_connections table with new tokens.
*/ refreshTokenIfNeeded( connection: SocialConnection ):
Promise<OAuthTokens>; /** * Validate content before scheduling. * Called
synchronously at schedule time – not at publish time. */
validateContent(post: ContentPost): ValidationResult;}
```

7.4 Platform-Specific Implementation Notes

Platform	Required Scopes	Content Types	File Limits	Char Limit	Rate Limit	Key Gotchas
Instagram Graph API	instagram_basic, instagram_content_publish, pages_read_engagement	Image, Video (Reel/Fee d), Carousel	Image: 8 MB; Video: 100 MB	Capti on: 2,200 chars	200 API calls/hou r per token	Requires Facebook Business account linked. Video uses a two-step container creation + publish flow. Carousel requires creating child containers first. Personal accounts are not supported.
Facebook Pages API	pages_manage_posts, pages_read_engagement, publish_pages	Link, Photo, Video	Photo: 10 MB; Video: 10 GB	63,206 chars	200 calls/hou r per token	Must use Page Access Token, not User token. Scheduled publishing via scheduled

Platform	Required Scopes	Content Types	File Limits	Char Limit	Rate Limit	Key Gotchas
						<code>_publish_time</code> (Unix timestamp, min 10 min in future, max 6 months). Video must be fully uploaded before scheduling.
X (Twitter) API v2	tweet.read, tweet.write, users.read, offline.access	Text, Image, Video, Poll	Image: 5 MB; GIF: 15 MB; Video: 512 MB	280 chars (basic) / 25,000 chars (Premium+)	50 posts/15 min per user (Basic tier)	OAuth2 with PKCE required. Media uploaded separately via v1.1 media upload endpoint (chunked for video). Check <code>processing_info</code> until video is ready before tweeting. Thread posting requires sequential API calls.
LinkedIn API	w_member_social, r_liteprofile, w_organization_social	Text, Article (link), Image, Video, Document	Image: 100 MB; Video: 5 GB (recommend <200 MB)	3,000 chars	Application-level: 500k calls/day	Company page posting requires Organization URN (not Person URN). Image posts use a

Platform	Required Scopes	Content Types	File Limits	Char Limit	Rate Limit	Key Gotchas
						multi-step upload: register → upload binary → finalize → post. Video requires polling for processing completion. Use v2 API only (v1 deprecated).
Pinterest API v5	boards:read, pins:write, boards:write	Image Pin, Video Pin, Carousel	Image: 32 MB; Video: 2 GB	Title: 100 chars; Description: 800 chars	10 requests/second per app	Board ID required on every pin creation. Alt text strongly recommended (accessibility). Video pin creation is a two-step upload: create upload → upload binary → create pin. Carousel pins require multiple media items.
TikTok Content Posting API	video.publish, video.upload, user.info.basic	Video only	Video: 4 GB; max 10 min	Caption: 2,200 chars; max 30	100 posts/day per user	Creator Marketplace access required for direct publishing. Video

Platform	Required Scopes	Content Types	File Limits	Char Limit	Rate Limit	Key Gotchas
				hashtags		length: 60s–10min for standard; 15s–60s for TikTok Stories. Support both DIRECT_POST and INBOX (draft to TikTok) modes. Chunked upload required for files >128 MB. Privacy setting must be specified.

7.5 Retry & Dead Letter Strategy

BullMQ handles retry scheduling automatically when `attempts` and `backoff` are configured on the queue. The delay formula is: $\text{delay} = 2^{\text{retry_count}} \times 30,000 \text{ ms}$.

A t t e m p t	D e l a y B e f o r e R e t r y	Cumulative Wait	
1 (i n i t i a l)	—	0s	
2 3 0 s (2 1 x 3 0)	30s		
3 6 0 s (2 2 x 3 0)	90s		

A t t e m p t	D e l a y	Cumulative Wait	
4	1	210s	
	2		
	0		
	s		
	(		
	2		
	s		
	x		
	3		
	0		
	)		
D	N	—	
e	o		
a	f		
d	u		
l	r		
e	t		
t	h		
t	e		
e	r		
r	r		
	e		
	t		
	r		
	y		

On final failure: set `publish_jobs.status = 'failed'`, store full error stack in `error_message`, emit `job:progress` WebSocket event, enqueue a `notification` job to alert the post creator and workspace managers. Manual retry is available via `POST`

`/api/v1/workspaces/:wid/jobs/:jid/retry`, which resets `retry_count = 0`, `status = 'pending'`, and re-enqueues the BullMQ job.

7.6 Time Zone Handling

- **Database:** All timestamps stored as `TIMESTAMPTZ` in UTC. PostgreSQL normalizes on insert.
- **Workspace default timezone:** Stored as an IANA tz string (e.g., `"America/New_York"`) in `workspaces.default_timezone`. Used as a fallback when the client does not send an offset.
- **API input:** Frontend sends `scheduled_at` as ISO 8601 with UTC offset (e.g., `"2026-09-15T14:00:00-04:00"`). The API validates with Zod's `z.string().datetime({ offset: true })` and stores the UTC-normalized value.
- **Scheduler:** The cron worker operates entirely in UTC. It never converts timezones internally.
- **API output:** All timestamps returned as UTC ISO 8601. The frontend converts to the workspace timezone for display using the `Intl.DateTimeFormat` API with the workspace's `default_timezone`.
- **DST safety:** Because all storage and scheduling is UTC, Daylight Saving Time transitions are transparent to the scheduling engine.

8. Media & Export Pipeline

8.1 Upload Flow

Large media files bypass the API server entirely using presigned URLs. The API server only handles metadata, never the file bytes for uploads.

```
// typescript// POST /api/v1/workspaces/:wid/media/presign// Body: { filename: string, mimeType: string, sizeBytes: number }import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';import { getSignedUrl } from '@aws-sdk/s3-request-presigner';import { v4 as uuidv4 } from 'uuid';export async function presignUpload(req, res) { const { filename, mimeType, sizeBytes } = req.body; const assetId = uuidv4(); const key = `workspaces/${req.workspace.id}/media/${assetId}/${filename}`; const command = new PutObjectCommand({ Bucket: process.env.R2_BUCKET_NAME, Key: key, ContentType: mimeType, ContentLength: sizeBytes, }); const uploadUrl = await getSignedUrl(s3Client, command, { expiresIn: 900 }); // 15 min //
```



```

Pre-create DB record in 'pending' state  await prisma.mediaAsset.create({
data: {      id: assetId,      workspace_id: req.workspace.id,
uploaded_by: req.user.id,      filename,      mime_type: mimeType,
size_bytes: sizeBytes,      storage_key: key,      status: 'pending', // add
this ephemeral status if needed    },  });  res.json({ data: { uploadUrl,
assetId, key }  });}
// typescript// workers/mediaWorker.tsimport sharp from 'sharp';import {
S3Client, GetObjectCommand, PutObjectCommand } from '@aws-sdk/client-s3';async
function processImage(assetId: string) {  const asset = await
prisma.mediaAsset.findUniqueOrThrow({ where: { id: assetId }  });  const stream
= await s3Client.send(new GetObjectCommand({    Bucket:
process.env.R2_BUCKET_NAME,    Key: asset.storage_key,  }));  const imageBuffer
= await streamToBuffer(stream.Body);  const metadata = await
sharp(imageBuffer).metadata();  // Generate thumbnail (300x300 cover crop,
WebP)  const thumbnail = await sharp(imageBuffer)    .resize(300, 300, { fit:
'cover'  })    .webp({ quality: 80  })    .withMetadata({ exif: {}  }) // strip
EXIF    .toBuffer();  const thumbnailKey = asset.storage_key.replace('/media/',
'/thumbnails/') + '.webp';  await s3Client.send(new PutObjectCommand({
Bucket: process.env.R2_BUCKET_NAME,    Key: thumbnailKey,    Body: thumbnail,
ContentType: 'image/webp',  }));  await prisma.mediaAsset.update({    where: {
id: assetId },    data: {      width: metadata.width,      height:
metadata.height,      thumbnail_key: thumbnailKey,    },  });}

```

8.2 Storage Configuration

```

# bash - Required environment variablesSTORAGE_PROVIDER=r2      # or
'replit'R2_ACCOUNT_ID=abc123xyzR2_ACCESS_KEY_ID=your_r2_access_keyR2_SECRET_ACC
ESS_KEY=your_r2_secret_keyR2_BUCKET_NAME=create-mediaR2_PUBLIC_URL=https://medi
a.yourdomain.com
// typescript// lib/storage.tsimport { S3Client } from
'@aws-sdk/client-s3';export const s3Client = new S3Client({  region: 'auto',
endpoint: `https://${process.env.R2_ACCOUNT_ID}.r2.cloudflarestorage.com`,
credentials: {    accessKeyId: process.env.R2_ACCESS_KEY_ID!,
secretAccessKey: process.env.R2_SECRET_ACCESS_KEY!,  },  // R2 does not support
path-style; force virtual-hosted-style  forcePathStyle: false,});export
function getPublicUrl(storageKey: string): string {  return
`${process.env.R2_PUBLIC_URL}/${storageKey}`;}

```

8.3 Content Export Pipeline

Three export formats are supported. All export endpoints require minimum Manager role. Exports are generated on-demand and streamed to the client.

Format	Endpoint	Method	Library	Timeout	Use Case
PDF	/workspaces/:wid/posts/:pid/export/pdf	GET	Puppeteer	30s	Client-facing content preview, approval docs
CSV	/workspaces/:wid/posts/export/csv	GET	csv-stringify	60s	Bulk metadata export, analytics ingestion

Format	Endpoint	Method	Library	Timeout	Use Case
JSON	/workspaces/:workspace_id/export	GET	Built-in	120s	Full workspace backup, migration between environments

```
// typescript// PDF export - stream responseimport puppeteer from
'puppeteer';export async function exportPostPDF(req, res) {  const post = await
prisma.contentPost.findUniqueOrThrow({    where: { id: req.params.pid },
include: { workspace: true },  });  const browser = await puppeteer.launch({
headless: true, args: ['--no-sandbox']  });  const page = await
browser.newPage();  await page.setContent(renderPostHTML(post), { waitUntil:
'networkidle0'  });  const pdf = await page.pdf({    format: 'A4',
printBackground: true,    margin: { top: '72px', right: '72px', bottom: '72px',
left: '72px' },  });  await browser.close();  res.setHeader('Content-Type',
'application/pdf');  res.setHeader('Content-Disposition', `attachment;
filename="post-${post.id}.pdf"`);  res.send(pdf);}
// typescript// CSV export - streamingimport { stringify } from
'csv-stringify';export async function exportPostsCSV(req, res) {
res.setHeader('Content-Type', 'text/csv');
res.setHeader('Content-Disposition', 'attachment; filename="posts.csv"');
const stringifier = stringify({  header: true,  columns: ['id', 'title',
'status', 'scheduled_at', 'published_at', 'target_platforms',
'tags', 'created_at'],  });  stringifier.pipe(res);  const posts = await
prisma.contentPost.findMany({    where: { workspace_id: req.workspace.id },
orderBy: { created_at: 'desc' },  });  for (const post of posts) {
stringifier.write({    ...post,    target_platforms:
post.target_platforms.join(','),    tags: post.tags.join(','),  });  }
stringifier.end();}
```

8.4 Image Optimization

The `sharp` library handles all image transformations. EXIF metadata is stripped before storing and before uploading to social platforms (privacy compliance). Platform-specific resize operations run in the publish worker immediately before uploading to the platform API.

Platform	Format	Width (px)	Height (px)	Aspect Ratio	Notes
Instagram	Feed Square	1080	1080	1:1	Minimum 320px; max 1440px
Instagram	Feed Portrait	1080	1350	4:5	Recommended for max screen real estate
Instagram	Story	1080	1920	9:16	Safe zone: center 1080×1420

Platform	Format	Width (px)	Height (px)	Aspect Ratio	Notes
Instagram	Reel Cover	1080	1920	9:16	Cover frame extracted from video
Facebook	Link Preview	1200	630	1.91:1	OG image standard
Facebook	Cover Photo	851	315	2.7:1	Desktop; mobile crops to 640×360
X (Twitter)	In-Stream Image	1600	900	16:9	Minimum 600×335; 5 MB limit
X (Twitter)	Summary Card	1200	628	1.91:1	OG image used by Twitter card
LinkedIn	Article Cover	1200	627	1.91:1	Standard OG dimensions
LinkedIn	Post Image	1200	628	1.91:1	Displayed in feed at ~552px wide
Pinterest	Standard Pin	1000	1500	2:3	Optimal; 600px min width
TikTok	Video Cover	1080	1920	9:16	Extracted frame; JPEG

9. Notifications System

9.1 Notification Types

Event	Recipient(s)	Channel(s)	Priority
Post published successfully	Post creator, Managers	In-app, Email (optional)	Normal
Post publish failed	Post creator, Managers	In-app, Email	High

Event	Recipient(s)	Channel(s)	Priority
Post approaching schedule (30 min warning)	Post creator	In-app	Normal
Member invited to workspace	Invitee	Email	High
Member joined workspace	Owners, Managers	In-app	Low
Social token expiring (within 7 days)	Managers, Owner	In-app, Email	High
Workspace plan limit approaching (80%)	Owner	In-app, Email	High
Post submitted for review	Managers	In-app	Normal
Post approved	Post creator	In-app	Normal

9.2 Delivery Channels

In-app notifications: Written to the `notifications` table and pushed via Socket.IO

`notification:new` event to the target user's session if connected. Users poll `GET /notifications` on reconnect to catch missed notifications.

Email: The `notification` BullMQ worker handles email delivery. Use Resend as the primary delivery provider with Nodemailer+SendGrid as fallback. Email templates are authored with React Email and compiled to static HTML at build time. Never build email HTML at runtime via string concatenation.

```
// typescript// notificationWorker.ts - email deliveryimport { Resend } from 'resend';import { PublishFailedEmail } from '@create/emails';const resend = new Resend(process.env.RESEND_API_KEY);async function sendPublishFailedEmail(payload: PublishFailedPayload) { await resend.emails.send({ from: 'CREATE <notifications@create.io>', to: payload.recipientEmail, subject: `Post publish failed: ${payload.postTitle}`, react: PublishFailedEmail({ ...payload }), });}
```

Outbound webhooks: If a workspace has configured a `webhook_url` (stored in `workspaces`), every notification event is also POSTed to that URL as a JSON payload. The request includes a `X-Create-Signature` header containing the HMAC-SHA256 signature of the raw body using the workspace's `webhook_secret`. Consumers must verify this signature before processing.

```
// typescriptimport { createHmac } from 'crypto';export function
signWebhookPayload(secret: string, body: string): string { return 'sha256=' +
createHmac('sha256', secret).update(body).digest('hex');}
```

9.3 Notifications Table Schema

```
-- sqlCREATE TYPE notification_type AS ENUM ( 'publish_success',
'publish_failed', 'schedule_warning', 'member_invited', 'member_joined',
'token_expiring', 'plan_limit', 'post_review_requested',
'post_approved');CREATE TABLE notifications ( id          UUID PRIMARY KEY
DEFAULT gen_random_uuid(), workspace_id UUID NOT NULL REFERENCES
workspaces(id) ON DELETE CASCADE, user_id          UUID NOT NULL REFERENCES
users(id) ON DELETE CASCADE, type          notification_type NOT NULL,
payload          JSONB NOT NULL DEFAULT '{}', -- type-specific data read_at
TIMESTAMPTZ, created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW());ALTER TABLE
notifications ENABLE ROW LEVEL SECURITY;CREATE POLICY
notifications_workspace_isolation ON notifications AS PERMISSIVE FOR ALL TO
app_user USING (workspace_id = current_setting('app.current_workspace_id',
true)::UUID);CREATE INDEX idx_notifications_user_read ON
notifications(user_id, read_at, created_at DESC) WHERE read_at IS NULL;
```

10. Replit Deployment Guide

10.1 Replit Project Structure

```
/├─ apps/├─ api/ # Express API server (Node 20 LTS)├─ src/├─ index.ts # Server entry point├─ routes/ # Route handlers grouped by resource├─ middleware/ # Auth, error, logging, rate-limit├─ lib/ # redis.ts, storage.ts, socket.ts├─ types/ # Express augmentation├─ package.json├─ tsconfig.json├─ web/ # React / Next.js frontend├─ src/├─ app/ # Next.js App Router pages├─ components/ # React components├─ lib/ # API client, socket client├─ package.json├─ next.config.js├─ packages/├─ db/ # Prisma schema + migrations├─ prisma/├─ schema.prisma├─ migrations/├─ src/├─ client.ts # Exported Prisma client singleton├─ queue/ # BullMQ worker definitions + job types├─ src/├─ queues.ts # Queue instances├─ publishWorker.ts├─ mediaWorker.ts├─ publishers/ # Platform-specific SocialPublisher impls├─ package.json├─ emails/ # React Email templates├─ shared/ # TypeScript types, utilities, constants├─ src/├─ types.ts├─ constants.ts├─ validators.ts # Zod schemas shared across apps├─ package.json├─ .replit # Replit run configuration├─ replit.nix # Nix environment (Node 20, ffmpeg, etc.)├─ package.json # Root workspace (npm workspaces)├─ docker-compose.yml # Local dev: Postgres, Redis, pgAdmin
```

10.2 .replit Configuration

```
# tomlrun = "npm run start:all"entrypoint = "apps/api/src/index.ts"[nix]channel = "stable-23_11"[deployment]run = ["sh", "-c", "npm run start:all"]deploymentTarget = "cloudrun"ignorePorts = false[[ports]]localPort = 3000externalPort = 80[[ports]]localPort = 3001externalPort = 3001[env]NODE_ENV = "production"PORT = "3000"
# replit.nix - system dependencies{ pkgs }: {  deps = [    pkgs.nodejs_20
pkgs.ffmpeg          # required by mediaWorker for video processing
pkgs.chromium         # required by Puppeteer for PDF export
pkgs.postgresql_16    # CLI tools only; DB is managed add-on    pkgs.redis
# CLI tools only; Redis is Upstash managed  ];  env = {
PUPPETEER_EXECUTABLE_PATH = "${pkgs.chromium}/bin/chromium";
PUPPETEER_SKIP_CHROMIUM_DOWNLOAD = "true";  };}
```

10.3 Environment Variables Reference

Variable	Required	Example Value	Description
DATABASE_URL	Yes	postgresql://user:password@host:5432/create_db	Prisma connection string to Replit PostgreSQL
REDIS_URL	Yes	rediss://default:token@host:6379	Upstash Redis connection string (TLS required)
JWT_SECRET	Yes	64-char random hex string	Signs access tokens; minimum 32 bytes
JWT_REFRESH_SECRET	Yes	64-char random hex string	Signs refresh tokens; different from JWT_SECRET
SOCIAL_TOKEN_ENCRYPTION_KEY	Yes	base64-encoded 32 random bytes	AES-256-GCM key for encrypting OAuth tokens at rest
STORAGE_PROVIDER	Yes	r2	'r2' or 'replit'; selects storage backend
R2_ACCOUNT_ID	If r2	abc123xyz789	Cloudflare account ID
R2_ACCESS_KEY_ID	If r2	cf_r2_key_...	R2 API access key
R2_SECRET_ACCESS_KEY	If r2	cf_r2_secret_...	R2 API secret key
R2_BUCKET_NAME	If r2	create-media	R2 bucket name
R2_PUBLIC_URL	If r2	https://media.yourdomain.com	Public CDN base URL for media delivery
RESEND_API_KEY	Yes	re_live_abc123	Resend email delivery API key

Variable	Required	Example Value	Description
FRONTEND_URL	Yes	https://app.create.io	Allowed CORS origin and cookie domain
NODE_ENV	Yes	production	'development', 'test', or 'production'
PORT	Yes	3000	API server listen port
LOG_LEVEL	No	info	winston log level: error, warn, info, debug
CORS_ORIGINS	No	https://app.create.io ,https://staging.create.io	Comma-separated additional allowed CORS origins
WEBHOOK_TOLERANCE_SECONDS	No	300	Max age of inbound webhook timestamp before rejection (anti-replay)
INSTAGRAM_APP_ID	If Instagram	123456789	Facebook App ID for Instagram Graph API
INSTAGRAM_APP_SECRET	If Instagram	fb_app_secret	Facebook App Secret
TWITTER_CLIENT_ID	If X/Twitter	twitter_oauth2_client_id	X API v2 OAuth2 Client ID
TWITTER_CLIENT_SECRET	If X/Twitter	twitter_oauth2_secret	X API v2 OAuth2 Client Secret
LINKEDIN_CLIENT_ID	If LinkedIn	linkedin_client_id	LinkedIn OAuth2 Client ID
LINKEDIN_CLIENT_SECRET	If LinkedIn	linkedin_secret	LinkedIn OAuth2 Client Secret
TIKTOK_CLIENT_KEY	If TikTok	tiktok_client_key	TikTok Content Posting API client key
TIKTOK_CLIENT_SECRET	If TikTok	tiktok_secret	TikTok Content Posting API client secret

10.4 Required Replit Add-ons

Add-on	Provider	Tier	Notes
PostgreSQL	Replit Database	Any (recommend dedicated for prod)	Enable in workspace settings → Database. Connection string injected as DATABASE_URL automatically.
Redis	Upstash via Replit integration	Free (10k cmd/day) for dev; Upstash Pro for production	Required for BullMQ and rate limiting. Free tier is insufficient for production queue workloads. Upstash Pro scales to millions of commands/day.
Object Storage	Replit Object Storage or Cloudflare R2	Replit: 1 GB free; R2: 10 GB free + \$0.015/GB/mo	R2 recommended for production due to S3 compatibility, egress pricing, and global CDN. Replit Object Storage acceptable for MVP.

10.5 Health Checks & Readiness

```
// typescript// apps/api/src/routes/health.tsimport { Router } from
'express';import { prisma } from '@create/db';import { redis } from
'../lib/redis';const router = Router();// Liveness: is the process alive?
Always 200.router.get('/health', (_req, res) => {  res.status(200).json({
status: 'ok', timestamp: new Date().toISOString() }));});// Readiness: can the
process serve traffic? Checks dependencies.router.get('/ready', async (_req,
res) => {  const checks: Record<string, 'ok' | 'error'> = {};  try {    await
prisma.$queryRaw`SELECT 1`;    checks.database = 'ok';  } catch {
checks.database = 'error';  }  try {    await redis.ping();    checks.redis =
'ok';  } catch {    checks.redis = 'error';  }  const allHealthy =
Object.values(checks).every(v => v === 'ok');  res.status(allHealthy ? 200 :
503).json({ status: allHealthy ? 'ready' : 'degraded', checks }));});export
default router;
```


NOTE

Replit Cloud Run deployments use the

`/ready`

endpoint for startup probes. Configure the probe path in your deployment settings. If

`/ready`

returns non-200, Replit will not route traffic to the instance and will attempt to restart it.

10.6 Database Migration on Deploy

```
// json - apps/api/package.json (scripts section){  "scripts": {    "build": "tsc",    "start": "node dist/index.js",    "dev": "tsx watch src/index.ts",    "postinstall": "prisma migrate deploy",    "migrate:dev": "prisma migrate dev",    "migrate:reset": "prisma migrate reset"  }}
```

`prisma migrate deploy` applies all pending migrations from `packages/db/prisma/migrations/` to the target database. It is non-interactive and safe to run in production — it never creates or rolls back; it only applies new migrations. The `postinstall` hook ensures migrations run on every `npm install`, which Replit triggers on each deploy.

WARNING

Never run

`prisma migrate dev`

in production. It can prompt for destructive operations and reset data. Only use

`prisma migrate deploy`

in any deployed environment.

10.7 Worker Process Management

```
// json - root package.json{  "scripts": {    "start:api": "cd apps/api && npm start",    "start:worker": "cd packages/queue && node dist/worker-entry.js",    "start:all": "concurrently \"npm run start:api\" \"npm run start:worker\" --names \"API,WORKER\" --prefix-colors \"blue,green\""}  }
// bash - install concurrentlynpm install -D concurrently
```

Scale-out pattern: When worker load increases, create a second Replit deployment (same codebase) that runs only `npm run start:worker` (not the API). Point it to the same `DATABASE_URL` and `REDIS_URL`. BullMQ distributes jobs across all connected worker instances automatically. The API and worker deployments are independent — scaling workers does not require scaling the API tier.

10.8 Secrets Management

All environment variables must be stored in the Replit Secrets panel (Settings → Secrets). Never commit a `.env` file or hardcode credentials in source. Validate all required secrets at startup using a Zod schema so the process fails fast with a clear error rather than crashing mid-request:

```
// typescript// apps/api/src/config/env.tsimport { z } from 'zod';const envSchema = z.object({  DATABASE_URL: z.string().url(),  REDIS_URL: z.string().url(),  JWT_SECRET: z.string().min(32, 'JWT_SECRET must be at least 32 characters'),  JWT_REFRESH_SECRET: z.string().min(32),  SOCIAL_TOKEN_ENCRYPTION_KEY: z.string().min(44, 'Must be 32-byte base64 string'),  STORAGE_PROVIDER: z.enum(['r2', 'replit']),  RESEND_API_KEY: z.string().startsWith('re_'),  FRONTEND_URL: z.string().url(),  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),  PORT: z.coerce.number().default(3000),  LOG_LEVEL: z.enum(['error', 'warn', 'info', 'debug']).default('info'),  WEBHOOK_TOLERANCE_SECONDS: z.coerce.number().default(300),});export const env = envSchema.parse(process.env);// Import 'env' from this module throughout the app - never access process.env directly.
```

11. Security Hardening Checklist

Each item must be verified before any public production deployment. Assign an owner for each item during sprint planning.

- **Helmet.js HTTP headers:** Install and configure `helmet()` as the first middleware. Enables CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy. Customize CSP to allow only known CDN origins.

- **CORS restricted to whitelist:** Set `origin` to the array from `FRONTEND_URL + CORS_ORIGINS`. Never use `origin: '*'`. Credentials mode requires explicit origin, not wildcard.
- **SQL injection: parameterized queries only:** Prisma parameterizes by default. Any raw SQL (e.g., RLS context setting) must use tagged template literals (`prisma.$queryRaw\`...\``), never string interpolation. Lint rule: `ban $executeRawUnsafe` except for SET LOCAL statements with UUID-validated inputs.
- **XSS: sanitize user-supplied HTML:** All rich text stored in `body_json` must be sanitized before storage using DOMPurify (server-side via `dompurify + jsdom`). Never render stored HTML without re-sanitizing in the frontend.
- **CSRF: double-submit cookie:** For all state-changing form endpoints, use the double-submit cookie pattern. Refresh token cookie is `SameSite=Strict; HttpOnly; Secure`. AJAX requests send a CSRF token in header matched against a cookie value.
- **HMAC webhook verification with replay protection:** All inbound webhooks at `/webhooks/:platform` must verify HMAC-SHA256 signature. Reject requests where the embedded timestamp is older than `WEBHOOK_TOLERANCE_SECONDS` (default: 300 seconds) to prevent replay attacks.
- **File upload: magic byte MIME validation:** Validate MIME type by reading the first 8 bytes (magic numbers) using the `file-type` npm package. Do not trust the `Content-Type` header or file extension alone. Enforce per-type size limits before processing.
- **File upload: virus scan stub:** Add a ClamAV scan step in the `media-process` worker before marking assets as ready. For MVP, implement the interface with a no-op and document the stub — full ClamAV integration is a required pre-launch item.
- **OAuth state parameter validation:** Generate a cryptographically random `state` value (32 bytes, hex) for every OAuth2 authorization code flow. Store state in a short-lived (10 min) Redis key. Reject callbacks where `state` does not match. This prevents CSRF on the OAuth flow.
- **Refresh token rotation on use:** Every successful `POST /auth/refresh` must revoke the submitted token and issue a new one. If a revoked token is detected being replayed, set `revoked_at` on all tokens for that user (token reuse detection = account breach signal).

- **Audit log all privileged actions:** Write an `audit_logs` record before (not after) any state change involving: member role changes, member removal, social connection add/remove, post approval, publish-now, workspace setting changes, and export actions. Use a DB transaction to make the audit write atomic with the action.
 - **Rate limiting on auth endpoints:** Apply `authLimiter` (5 req/min per IP) specifically to `POST /auth/login`, `POST /auth/register`, and `POST /auth/accept-invite`. Standard `authenticatedLimiter` (200 req/min) applies to all other authenticated routes.
 - **RLS enforced at DB layer:** Confirm RLS is active by running `SELECT relrowsecurity FROM pg_class WHERE relname = 'content_posts' → must return true`. Write an integration test that asserts a query with workspace A's context cannot return workspace B's rows.
 - **Encrypt social OAuth tokens at rest:** All `access_token` and `refresh_token` values in `social_connections` must be encrypted with AES-256-GCM before INSERT/UPDATE and decrypted after SELECT. Never log decrypted token values.
 - **Dependency scanning: npm audit in CI:** Add `npm audit --audit-level=moderate` as a required CI step. Any moderate or higher severity vulnerability fails the build until resolved or explicitly suppressed with justification.
 - **Secrets never logged:** Configure the Winston logger to redact any log entries containing fields named `password`, `token`, `access_token`, `refresh_token`, `secret`, or `authorization`. Use a custom formatter or the `winston-sanitize` plugin.
 - **HTTPS only:** Replit Cloud Run enforces TLS termination. Ensure the Express app sets `app.set('trust proxy', 1)` to correctly read forwarded IP addresses. Never serve mixed content.
 - **Email verification before publishing:** Require `users.email_verified_at IS NOT NULL` before a user can publish content or connect social integrations. Send verification email on registration; resend via `POST /auth/resend-verification`.
-

12. Migration Path: v1 (Single-Operator) → v2 (Multi-Tenant)

MIGRATION PRINCIPLE

At no point during the migration should existing content be inaccessible or existing functionality be broken. The v1 operator continues using the app normally throughout the migration. Each phase is independently deployable and rollback-safe.

12.1 Phase 1 – Foundation (Week 1–2)

Goal: Add the multi-tenant schema alongside the existing data without any functional changes.

7. Create the seed workspace: `INSERT INTO workspaces (id, name, slug, ...) VALUES ('00000000-0000-0000-0000-000000000001', 'Default Workspace', 'default', ...)`
8. Create the seed admin user from the existing operator credentials. Hash the existing password with bcrypt if not already hashed.
9. Add `workspace_id` column (nullable) to all existing content tables. Do not enforce NOT NULL yet.
10. Run a backfill: `UPDATE content_posts SET workspace_id = '00000000-0000-0000-0000-000000000001' WHERE workspace_id IS NULL`
11. Once backfill is confirmed complete, make `workspace_id` NOT NULL and add the FK constraint.
12. Add `users`, `workspaces`, `workspace_members` tables.
13. Insert the operator as a workspace member with role `owner`.
14. Replace existing session auth with JWT middleware. Issue the operator a JWT on next login.
Existing functionality is unchanged — the operator now logs in with email/password and gets a JWT instead of a session cookie.

```
-- sql-- Phase 1 seed migrationINSERT INTO workspaces (id, name, slug, plan, owner_user_id)VALUES ( '00000000-0000-0000-0000-000000000001', 'Default Workspace', 'default', 'pro', (SELECT id FROM users WHERE email = 'admin@example.com'));INSERT INTO workspace_members (workspace_id, user_id, role)VALUES ( '00000000-0000-0000-0000-000000000001', (SELECT id FROM users WHERE email = 'admin@example.com'), 'owner');
```

12.2 Phase 2 – Isolation (Week 3–4)

Goal: Enable true tenant isolation and secure credential storage.

15. Enable RLS on all workspace-scoped tables (start with permissive SELECT policy, then add INSERT/UPDATE/DELETE policies).
16. Switch application DB connection to the low-privilege `app_user` role.
17. Create `social_connections` table. Migrate existing plaintext social credentials from environment variables into the table with encryption. Remove the platform credentials from env vars after confirming migration.
18. Create `refresh_tokens` table. Begin storing refresh tokens on login.
19. Deploy BullMQ worker. Convert any existing cron-based scheduling to queue-based jobs. The old cron continues running in parallel until the queue is confirmed stable.
20. Disable the old cron after 48 hours of confirmed queue operation.

12.3 Phase 3 – Publishing Engine (Week 5–6)

Goal: Implement the full social publishing pipeline.

21. Implement `SocialPublisher` for each required platform in priority order (highest-used platform first). Each publisher is covered by unit tests before deployment.
22. Deploy presigned media upload flow. Migrate existing local media files to R2/Object Storage. Update `media_assets` records with `storage_key` values.
23. Deploy media-process worker. Run it against all existing media assets to generate thumbnails and extract metadata.
24. Enable Socket.IO. Deploy frontend WebSocket client. Real-time updates are additive — clients that don't connect to the socket still get updates on next page load.
25. Enable workspace invitation flow.

12.4 Phase 4 – Polish & Hardening (Week 7–8)

Goal: Production-readiness, observability, and security sign-off.

26. Deploy notifications system (in-app + email).
27. Enable audit logging on all privileged endpoints.
28. Complete security hardening checklist (Section 11). Each item gets a GitHub issue; all must be closed before launch.
29. Run load test: `k6 run k6/load-test.js` targeting 100 concurrent users. Accept criteria: p95 API response time < 300ms; error rate < 0.1%; no DB connection pool exhaustion.
30. Review and rotate all secrets. Confirm no secrets in source control via `git log -S 'secret_value'`.
31. Documentation pass: update README with new env vars, migration instructions, and worker setup.

12.5 Rollback Strategy

- **Database migrations:** Every migration has a corresponding down migration in `packages/db/migrations/down/`. Rollback command: `prisma migrate resolve --rolled-back <migration_name>` followed by manual execution of the down SQL file.
 - **Feature flags:** A `feature_flags` table (key TEXT PRIMARY KEY, enabled BOOLEAN, workspace_id UUID nullable) allows disabling the new publishing engine without a redeploy. Workers check the flag before processing. The API falls back to the legacy export flow if the flag is off.
 - **Blue/green deployment:** If Replit deployment budget allows, maintain two deployment slots: `production` (current stable) and `staging` (next release). Traffic can be switched back to the previous slot within seconds via the Replit dashboard if a critical bug is detected post-deploy.
 - **Data backups:** Configure daily automated PostgreSQL backups. Before each phase migration, trigger a manual backup and verify restore-ability in a separate environment.
-

13. Testing Strategy

13.1 Unit Tests

Unit tests cover all business logic that does not require a database or network. Every `SocialPublisher` implementation, every token utility function, every Zod validator, and every BullMQ job processor has unit test coverage.

- **Framework:** Jest 29+ with `ts-jest` transformer.
- **Mocking:** Mock `@prisma/client` and `ioredis` using Jest's module mock system. Use `jest.mock('../lib/redis')` at the top of each test file that touches queue or cache logic.
- **Coverage target:** 80% line coverage for `packages/queue/src/publishers/` and `apps/api/src/middleware/`.
- **Key test cases:** Token encryption/decryption round-trip; `enforceRole` grants and denies for each role pair; publisher `validateContent` catches character limit violations; retry backoff delay calculation; presigned URL key generation is workspace-scoped.

```
// typescript// Example: token encryption round-trip testimport { encryptToken, decryptToken } from '../src/lib/crypto';process.env.SOCIAL_TOKEN_ENCRYPTION_KEY = Buffer.from('a'.repeat(32)).toString('base64');describe('Token Encryption', () => { it('round-trips a token correctly', () => { const original = 'ya29.abcl23_oauth_access_token'; const encrypted = encryptToken(original); expect(encrypted).not.toEqual(original); const decrypted = decryptToken(encrypted); expect(decrypted).toEqual(original); }); it('produces unique ciphertexts for same input (IV randomness)', () => { const t1 = encryptToken('same-token'); const t2 = encryptToken('same-token'); expect(t1).not.toEqual(t2); expect(decryptToken(t1)).toEqual(decryptToken(t2)); });});
```

13.2 Integration Tests

Integration tests run against a real PostgreSQL instance with RLS enabled. They test the full request-response cycle including middleware, database writes, and RLS isolation.

- **Framework:** Supertest against the Express app instance. Tests import the app directly, not a running server.
- **Database:** Docker PostgreSQL 16 container in CI. Seeded fresh before each test suite using `@faker-js/faker` factories.

- **Key RLS isolation test:**

```
// typescript// Integration test: RLS workspace isolationimport request from 'supertest';import { app } from '../src/index';import { createTestWorkspace, createTestUser, issueTestToken } from './factories';describe('RLS: workspace isolation', () => { it('prevents workspace A from reading workspace B posts', async () => { const workspaceA = await createTestWorkspace(); const workspaceB = await createTestWorkspace(); const userA = await createTestUser(workspaceA.id, 'owner'); const tokenA = issueTestToken(userA); // Create a post in workspace B await prisma.contentPost.create({ data: { workspace_id: workspaceB.id, title: 'Workspace B Secret Post', created_by: workspaceB.ownerId, body_json: {}, target_platforms: [], }, }); // Attempt to list posts using workspace A's token against workspace A's endpoint const res = await request(app).get(`/api/v1/workspaces/${workspaceA.id}/posts`).set('Authorization', `Bearer ${tokenA}`).expect(200); const postTitles = res.body.data.map((p: any) => p.title); expect(postTitles).not.toContain('Workspace B Secret Post'); });});
```

13.3 End-to-End Tests

E2E tests cover complete user journeys through the browser using Playwright. Social platform API calls are intercepted by MSW (Mock Service Worker) at the network level — no real API calls are made in E2E tests.

- **Framework:** Playwright 1.40+, targeting Chromium.
- **MSW:** Run as a Node.js server handler in the Playwright test environment to intercept fetch calls to social platform domains.
- **Critical journeys to cover:**

Signup → verify email → create workspace → invite member → member accepts

Create draft post → editor adds media → manager approves → schedule → confirm publish job created

Connect Instagram integration (OAuth mock) → create post targeting Instagram → verify publish job status transitions

Failed publish → retry → success

Export post as PDF → download file

13.4 CI on Replit (GitHub Actions)

```
# yaml - .github/workflows/ci.ymlname: CIon: push: branches: [main,
develop] pull_request: branches: [main]jobs: test: runs-on:
ubuntu-latest timeout-minutes: 30 services: postgres: image:
postgres:16-alpine env: POSTGRES_USER: test
POSTGRES_PASSWORD: test POSTGRES_DB: create_test ports:
['5432:5432'] options: >- --health-cmd pg_isready
--health-interval 10s --health-timeout 5s --health-retries 5
redis: image: redis:7-alpine ports: ['6379:6379'] options:
--health-cmd "redis-cli ping" --health-interval 10s env: DATABASE_URL:
postgresql://test:test@localhost:5432/create_test REDIS_URL:
redis://localhost:6379 JWT_SECRET:
test_jwt_secret_at_least_32_chars_long_here JWT_REFRESH_SECRET:
test_refresh_secret_at_least_32_chars_long SOCIAL_TOKEN_ENCRYPTION_KEY:
dGVzdC1rZXktMzItYn10ZXMtOG9uZy1mb3ItYWVzMjU2 STORAGE_PROVIDER: replit
RESEND_API_KEY: re_test_placeholder FRONTEND_URL: http://localhost:3000
NODE_ENV: test steps: - uses: actions/checkout@v4 - name: Setup
Node.js 20 uses: actions/setup-node@v4 with:
node-version: '20' cache: 'npm' - name: Install dependencies
run: npm ci - name: Run Prisma migrations on test DB run: npm run
-w packages/db migrate:deploy env: DATABASE_URL:
postgresql://test:test@localhost:5432/create_test - name: Run unit tests
run: npm run test:unit --workspaces --if-present - name: Run integration
tests run: npm run test:integration --workspaces --if-present -
name: Install Playwright browsers run: npx playwright install
--with-deps chromium - name: Build web app for E2E run: npm run
build -w apps/web - name: Run E2E tests run: npx playwright test
- name: npm audit run: npm audit --audit-level=moderate - name:
Upload Playwright report uses: actions/upload-artifact@v4 if:
failure() with: name: playwright-report path:
playwright-report/ retention-days: 7
```

Appendix A: Glossary

Term	Definition
Workspace	The primary tenant unit in CREATE. Represents one team or brand account. All content, media, social connections, and members belong to exactly one workspace. The billing unit for SaaS subscriptions.
Member Role	An authorization level assigned to a user within a specific workspace. Roles are: Viewer, Creator, Editor, Manager, Owner. A user may hold different roles in different workspaces simultaneously.
Publish Job	A record in the <code>publish_jobs</code> table representing a single attempt to publish a <code>content_post</code> to a specific social platform. One content post targeting three platforms produces three independent publish jobs.
Content Post	A record in the <code>content_posts</code> table representing a piece of social media content in any lifecycle state (draft through published). Contains rich text body, media references, scheduling metadata, and platform targets.

Term	Definition
OAuth Token Vault	The <code>social_connections</code> table, which stores per-workspace, per-platform OAuth access and refresh tokens encrypted with AES-256-GCM. The "vault" abstraction isolates token storage and decryption from the rest of the application.
Dead Letter Queue	The state of a BullMQ job that has exhausted all retry attempts. In CREATE, a dead-lettered publish job has <code>status = 'failed'</code> and <code>retry_count = 4</code> . It remains in the database for audit purposes and can be manually retried via the API.
RLS (Row-Level Security)	A PostgreSQL feature that restricts which rows a database role can SELECT, INSERT, UPDATE, or DELETE. In CREATE, RLS policies on workspace-scoped tables enforce tenant isolation at the database layer, independent of API logic.
BullMQ	A Node.js job queue library backed by Redis. Used in CREATE for the <code>content-publish</code> , <code>media-process</code> , and <code>notification</code> queues. Provides durable job storage, retry logic, delayed jobs, and concurrency control.
Presigned URL	A time-limited, pre-authorized URL generated by the API server that allows a client to upload a file directly to object storage (R2 or Replit Object Storage) without routing the file bytes through the API server. Generated using the AWS SDK <code>getSignedUrl</code> function.
PKCE (Proof Key for Code Exchange)	An OAuth2 extension (RFC 7636) that prevents authorization code interception attacks. Required for public OAuth clients (SPAs, mobile apps). The client generates a random <code>code_verifier</code> , sends its SHA-256 hash (<code>code_challenge</code>) in the authorization request, and proves possession of the verifier when exchanging the code for tokens.

Appendix B: Dependency Manifest

All packages pinned to their minimum compatible version at time of blueprint authorship. Pin exact versions in `package-lock.json`; use caret ranges in `package.json` only for patch-level updates. Run `npm audit` before every production deploy.

Package	Pinned Version	Purpose
Core API		
<code>express</code>	<code>^4.19.2</code>	HTTP server framework

Package	Pinned Version	Purpose
helmet	^7.1.0	HTTP security headers middleware
cors	^2.8.5	CORS policy middleware
express-rate-limit	^7.3.1	Request rate limiting
rate-limit-redis	^4.2.0	Redis store for express-rate-limit
Database & ORM		
prisma	^5.16.0	ORM CLI, migration engine
@prisma/client	^5.16.0	Generated Prisma DB client
Authentication		
passport	^0.7.0	Authentication middleware framework
passport-local	^1.0.0	Email/password auth strategy
passport-google-oauth20	^2.0.0	Google OAuth2 strategy
jsonwebtoken	^9.0.2	JWT sign and verify
bcrypt	^5.1.1	Password and token hashing
Validation & Schema		
zod	^3.23.8	Runtime type validation, env schema, request body parsing
Queue & Cache		
bullmq	^5.12.0	Redis-backed job queue
ioredis	^5.4.1	Redis client for BullMQ and rate limiting
node-cron	^3.0.3	Cron scheduler for polling scheduled posts
Media Processing		
sharp	^0.33.4	High-performance image resize, format conversion, EXIF strip

Package	Pinned Version	Purpose
@aws-sdk/client-s3	^3.600.0	S3-compatible client for R2 and object storage
@aws-sdk/s3-request-presigner	^3.600.0	Presigned URL generation
multer	^1.4.5-lts.1	Multipart form-data parsing (for direct-to-API fallback uploads)
file-type	^19.3.0	MIME type detection by magic bytes
puppeteer	^22.12.0	Headless Chrome for PDF export
csv-stringify	^6.5.1	Streaming CSV serialization for post export
Real-Time		
socket.io	^4.7.5	WebSocket server with workspace-scoped rooms
Email		
resend	^3.4.0	Transactional email delivery API
react-email	^2.1.4	React-based email template authoring
Logging & Observability		
winston	^3.13.0	Structured logging with configurable transports
Process Management		
concurrently	^8.2.2	Run API + worker processes in one Replit deployment unit
Testing		
jest	^29.7.0	Unit and integration test runner
ts-jest	^29.2.3	TypeScript transformer for Jest
supertest	^7.0.0	HTTP integration testing for Express

Package	Pinned Version	Purpose
@playwright/test	^1.45.0	End-to-end browser testing
msw	^2.3.1	Mock Service Worker for intercepting social API calls in tests
@faker-js/faker	^8.4.1	Test data factories for seeding
TypeScript		
typescript	^5.5.3	TypeScript compiler
tsx	^4.16.2	TypeScript execute for development (replaces ts-node)
@types/express	^4.17.21	Express TypeScript types
@types/node	^20.14.0	Node.js built-in TypeScript types
@types/jsonwebtoken	^9.0.6	JWT TypeScript types
@types/bcrypt	^5.0.2	bcrypt TypeScript types
@types/passport	^1.0.16	Passport TypeScript types
@types/multer	^1.4.11	Multer TypeScript types